

COMP 1324 Algorithmics

Shortest Path

Hikmat Farhat

Single Source Shortest Path

- Given a graph $G = (V, E)$ with a real-valued weight function $w : E \rightarrow \mathbb{R}$ we often ask the question:
- What is the minimal cost (shortest) path from $s \in V$ to all other vertices of the graph.
- We will look at two algorithms that perform that task
 - 1 Bellman-Ford.
 - 2 Dijkstra.
- First we need some definitions and theorems.

- Let $G = (V, E)$ with the associated weight function $w : E \rightarrow \mathbb{R}$.
- The total weight of a path $p = (v_0, \dots, v_k)$ is the sum of weights of individual edges:

$$w(p) = \sum_{i=1}^k w(v_{i-1}, v_i)$$

- The shortest path cost δ

$$\delta(u, v) = \begin{cases} \min_p w(p) & \text{if there at least one path from } u \text{ to } v \\ \infty & \text{otherwise} \end{cases}$$

Negative weights and cycles

- Even if a path contains edges with negative weight a shortest path can still be defined.
- It is undefined if the path contains a negative weight **cycle**.
- This is because we can "cross" the cycle as many times as we want, every time lower the cost.
- Therefore in the case when there is a negative cycle on a path from u to v then we set $\delta(u, v) = -\infty$ where $\delta(a, b)$ is the shortest path cost from a to b .

Example of Negative Cycles

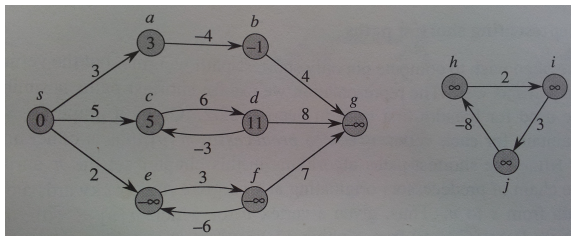


Figure: from CLRS

- $\delta(s, a) = 3, \delta(s, b) = -1, \delta(s, c) = 5, \delta(s, d) = 11$.
- (e, f) form a negative cycle therefore any node reachable from s through this cycle has $\delta = -\infty$
 $\delta(s, e) = \delta(s, f) = \delta(s, g) = -\infty$
- h, i, j are not reachable from s thus
 $\delta(s, h) = \delta(s, i) = \delta(s, j) = \infty$

Representation of Shortest Paths

- In all the algorithms that we will deal with, we maintain for every vertex v its predecessor $v.\pi$ (which could be NULL)
- At **termination** $v.\pi$ will be the predecessor of v on a shortest path from source s to v .
- We also maintain a value $v.\delta$ which at termination will be the value of the shortest path cost from source s to v .
- During the execution of the algorithm $v.\delta$ will be **an upper bound** on the value of the shortest path cost.

Relaxation

- **Relaxing** an edge (u, v) means testing if we can improve the shortest path cost of v by using the edge (u, v) .
- If we can then we update $v.\delta$ and $v.\pi$.

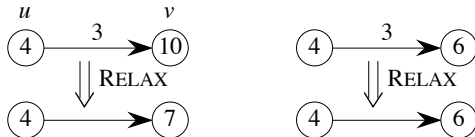


Figure: from CLRS

- In the figure to the left the cost of v was changed to the new cost (7) whereas to the right it was not changed since the new cost (7) is bigger than the current (6).
- What is NOT shown is the change to $v.\pi$ in the first case

Initialization and Relaxation

- Initially all vertices (except the source) have cost ∞ and no predecessors (including the source).

```
function INITIALIZE( $G, s$ )
```

```
    foreach  $v \in V$  do
```

```
         $v.\delta \leftarrow \infty$ 
```

```
         $v.\pi \leftarrow \text{NULL}$ 
```

```
     $s.\delta \leftarrow 0$ 
```

```
function RELAX( $u, v$ )
```

```
    if  $v.\delta > u.\delta + w(u, v)$  then
```

```
         $v.\delta \leftarrow u.\delta + w(u, v)$ 
```

```
         $v.\pi \leftarrow u$ 
```

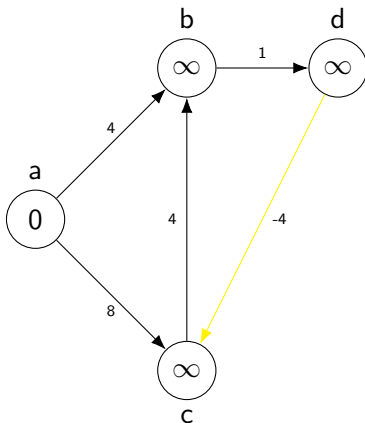
Bellman-Ford Algorithm

- The Bellman-Ford algorithm computes the shortest path from a given source to all other nodes in the graph.
- It works with negative weights and can detect negative cycles.
- It uses the function RELAX to compute the shortest path.
- RELAX is applied to all edges in the graph $|V| - 1$ times.

Example

First iteration

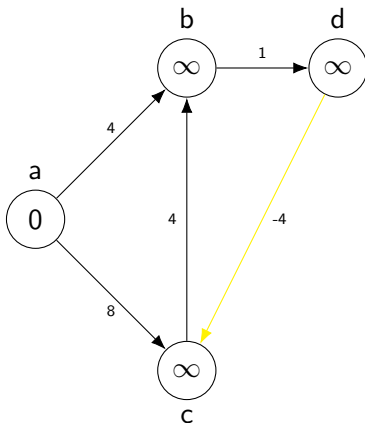
Consider edges in order $\{(d,c), (c,b), (b,d), (a,c), (a,b)\}$



Node	δ	π
a	0	NIL
b	∞	NIL
c	∞	NIL
d	∞	NIL

Example

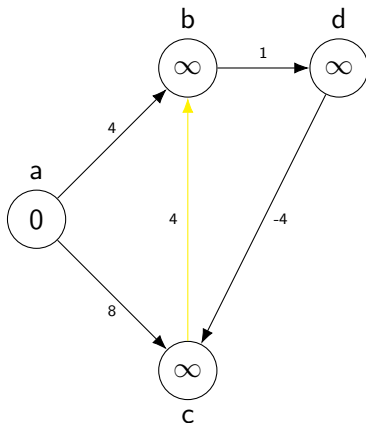
Consider edges in order $\{(d,c), (c,b), (b,d), (a,c), (a,b)\}$



Node	δ	π
a	0	NIL
b	∞	NIL
c	∞	NIL
d	∞	NIL

Example

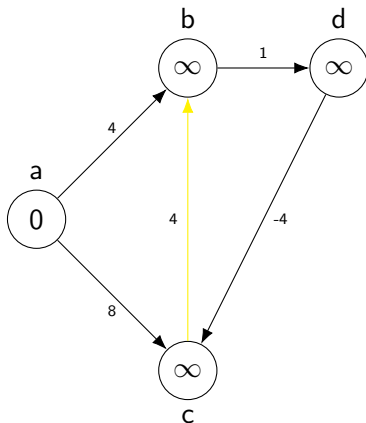
Consider edges in order $\{(d,c), (c,b), (b,d), (a,c), (a,b)\}$



Node	δ	π
a	0	NIL
b	∞	NIL
c	∞	NIL
d	∞	NIL

Example

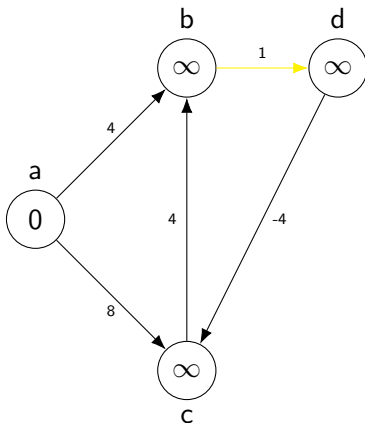
Consider edges in order $\{(d,c), (c,b), (b,d), (a,c), (a,b)\}$



Node	δ	π
a	0	NIL
b	∞	NIL
c	∞	NIL
d	∞	NIL

Example

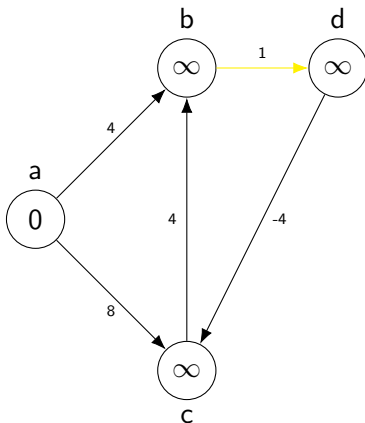
Consider edges in order $\{(d,c),(c,b), (b,d),(a,c),(a,b)\}$



Node	δ	π
a	0	NIL
b	∞	NIL
c	∞	NIL
d	∞	NIL

Example

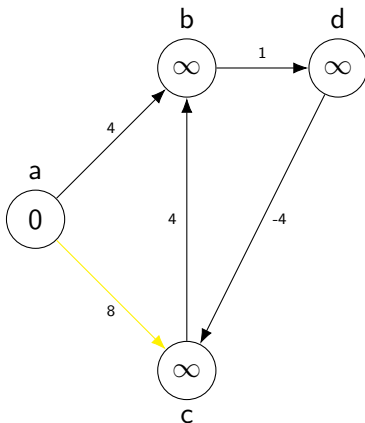
Consider edges in order $\{(d,c),(c,b), (b,d),(a,c),(a,b)\}$



Node	δ	π
a	0	NIL
b	∞	NIL
c	∞	NIL
d	∞	NIL

Example

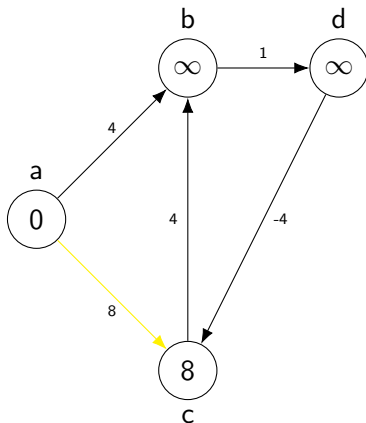
Consider edges in order $\{(d,c),(c,b),(b,d), \mathbf{(a,c)},(a,b)\}$



Node	δ	π
a	0	NIL
b	∞	NIL
c	∞	NIL
d	∞	NIL

Example

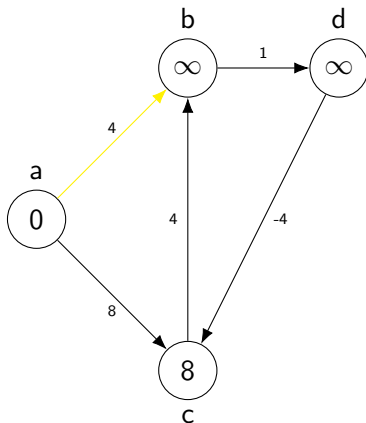
Consider edges in order $\{(d,c),(c,b),(b,d), \mathbf{(a,c)}, (a,b)\}$



Node	δ	π
a	0	NIL
b	∞	NIL
c	8	a
d	∞	NIL

Example

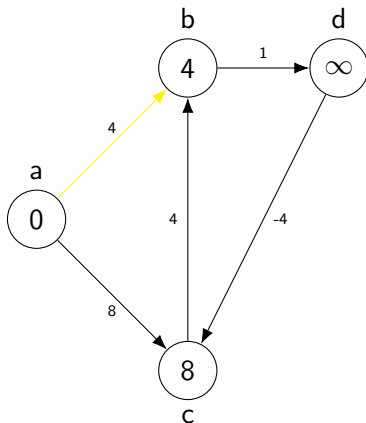
Consider edges in order $\{(d,c),(c,b),(b,d),(a,c), (a,b)\}$



Node	δ	π
a	0	NIL
b	∞	NIL
c	8	a
d	∞	NIL

Example

Consider edges in order $\{(d,c),(c,b),(b,d),(a,c), (a,b)\}$

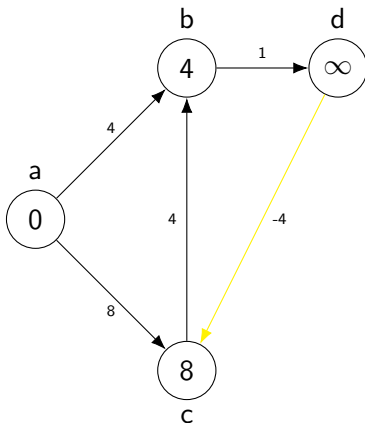


Node	δ	π
a	0	NIL
b	4	a
c	8	a
d	∞	NIL

Example

Second iteration

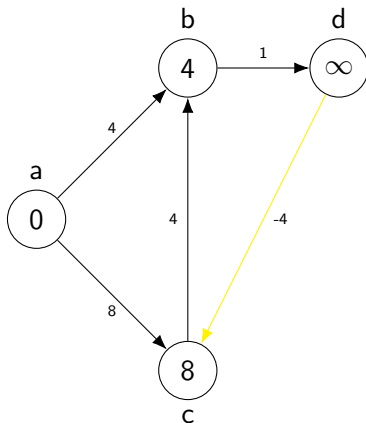
Consider edges in order $\{(d,c), (c,b), (b,d), (a,c), (a,b)\}$



Node	δ	π
a	0	NIL
b	4	a
c	8	a
d	∞	NIL

Example

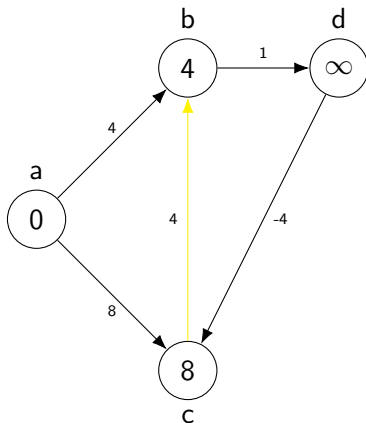
Consider edges in order $\{(d,c), (c,b), (b,d), (a,c), (a,b)\}$



Node	δ	π
a	0	NIL
b	4	a
c	8	a
d	∞	NIL

Example

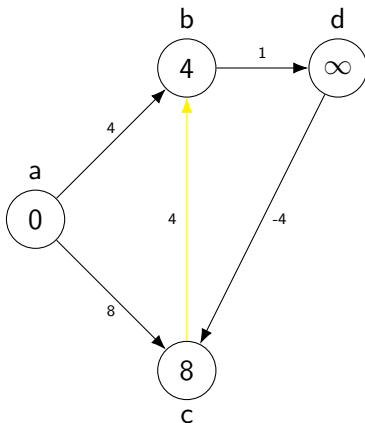
Consider edges in order $\{(d,c), (c,b), (b,d), (a,c), (a,b)\}$



Node	δ	π
a	0	NIL
b	4	a
c	8	a
d	∞	NIL

Example

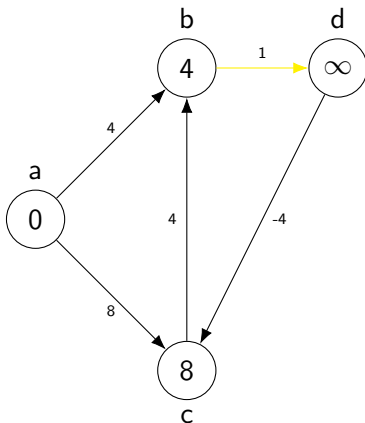
Consider edges in order $\{(d,c), (c,b), (b,d), (a,c), (a,b)\}$



Node	δ	π
a	0	NIL
b	4	a
c	8	a
d	∞	NIL

Example

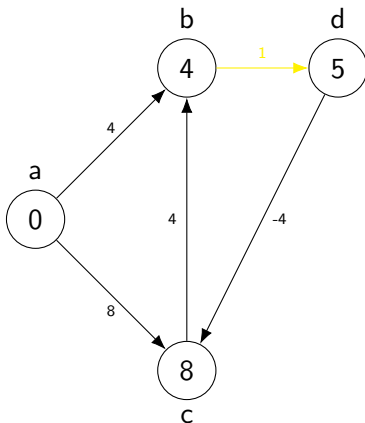
Consider edges in order $\{(d,c),(c,b), (b,d), (a,c), (a,b)\}$



Node	δ	π
a	0	NIL
b	4	a
c	8	a
d	∞	NIL

Example

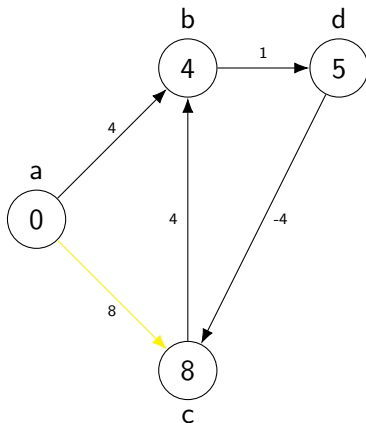
Consider edges in order $\{(d,c),(c,b),(b,d),(a,c),(a,b)\}$



Node	δ	π
a	0	NIL
b	4	a
c	8	a
d	5	b

Example

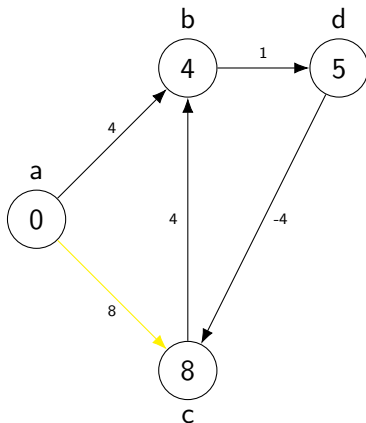
Consider edges in order $\{(d,c),(c,b),(b,d), \mathbf{(a,c)}, (a,b)\}$



Node	δ	π
a	0	NIL
b	4	a
c	8	a
d	5	b

Example

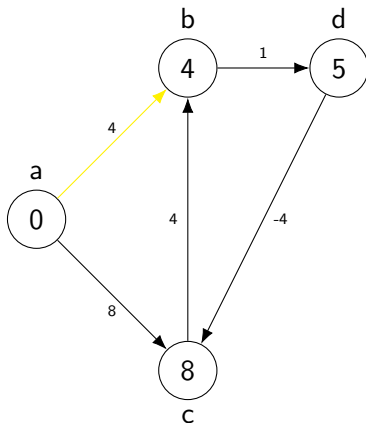
Consider edges in order $\{(d,c),(c,b),(b,d), \mathbf{(a,c)}, (a,b)\}$



Node	δ	π
a	0	NIL
b	4	a
c	8	a
d	5	b

Example

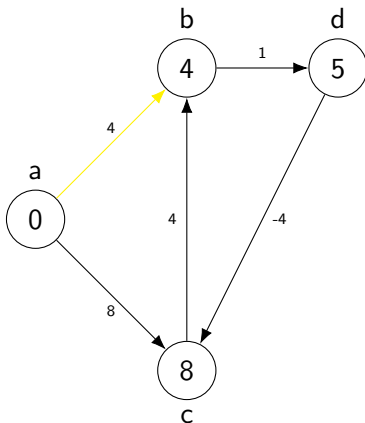
Consider edges in order $\{(d,c),(c,b),(b,d),(a,c), (a,b)\}$



Node	δ	π
a	0	NIL
b	4	a
c	8	a
d	5	b

Example

Consider edges in order $\{(d,c),(c,b),(b,d),(a,c), (a,b)\}$

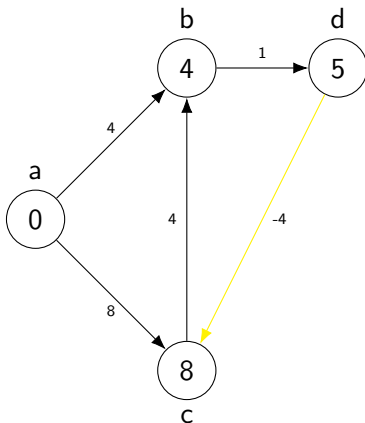


Node	δ	π
a	0	NIL
b	4	a
c	8	a
d	5	b

Example

Third iteration

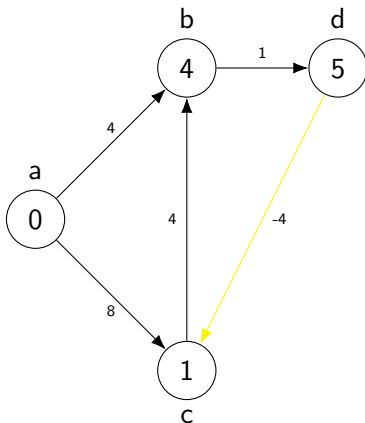
Consider edges in order $\{(d,c), (c,b), (b,d), (a,c), (a,b)\}$



Node	δ	π
a	0	NIL
b	4	a
c	8	a
d	5	b

Example

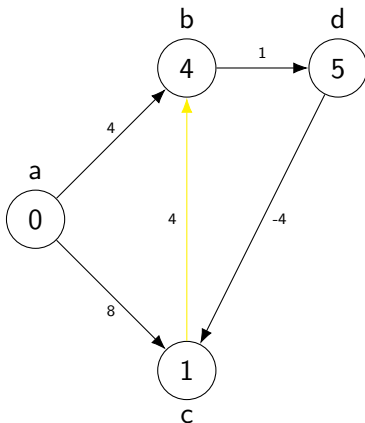
Consider edges in order $\{(d,c), (c,b), (b,d), (a,c), (a,b)\}$



Node	δ	π
a	0	NIL
b	4	a
c	1	d
d	5	b

Example

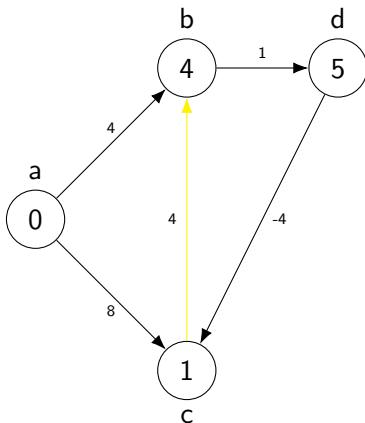
Consider edges in order $\{(d,c), (c,b), (b,d), (a,c), (a,b)\}$



Node	δ	π
a	0	NIL
b	4	a
c	1	d
d	5	b

Example

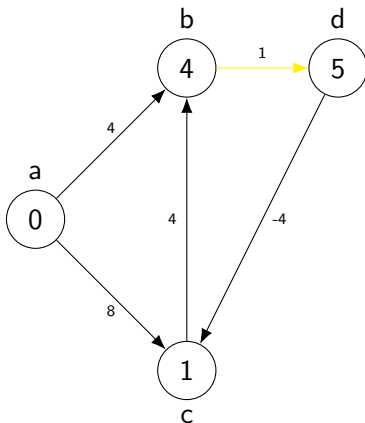
Consider edges in order $\{(d,c), (c,b), (b,d), (a,c), (a,b)\}$



Node	δ	π
a	0	NIL
b	4	a
c	1	d
d	5	b

Example

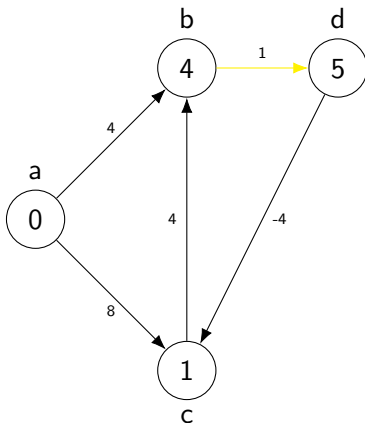
Consider edges in order $\{(d,c),(c,b), (b,d), (a,c), (a,b)\}$



Node	δ	π
a	0	NIL
b	4	a
c	1	d
d	5	b

Example

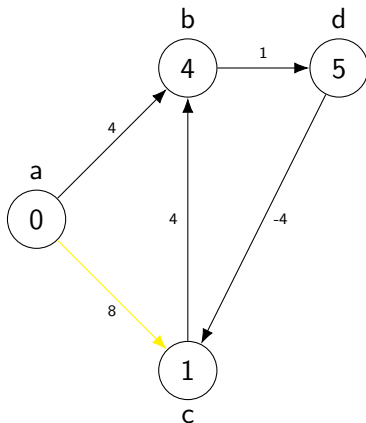
Consider edges in order $\{(d,c),(c,b),(b,d),(a,c),(a,b)\}$



Node	δ	π
a	0	NIL
b	4	a
c	1	d
d	5	b

Example

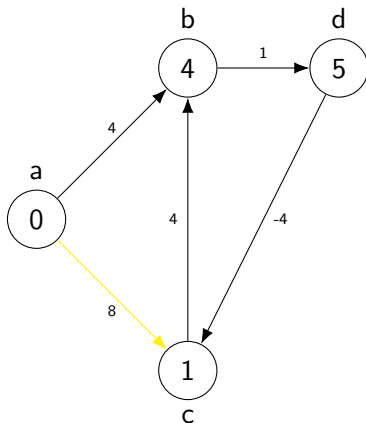
Consider edges in order $\{(d,c),(c,b),(b,d), \mathbf{(a,c)}, (a,b)\}$



Node	δ	π
a	0	NIL
b	4	a
c	1	d
d	5	b

Example

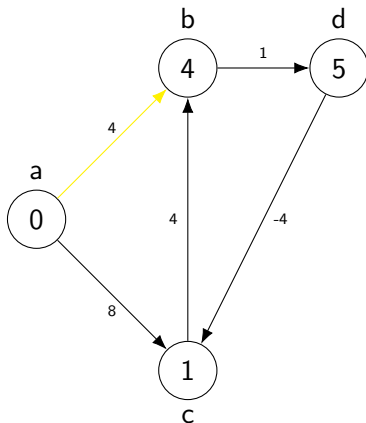
Consider edges in order $\{(d,c),(c,b),(b,d), \mathbf{(a,c)}, (a,b)\}$



Node	δ	π
a	0	NIL
b	4	a
c	1	d
d	5	b

Example

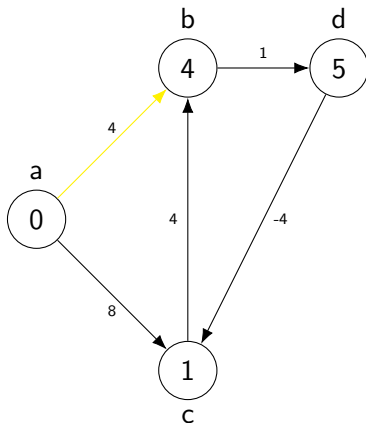
Consider edges in order $\{(d,c),(c,b),(b,d),(a,c), (a,b)\}$



Node	δ	π
a	0	NIL
b	4	a
c	1	d
d	5	b

Example

Consider edges in order $\{(d,c),(c,b),(b,d),(a,c), (a,b)\}$



Node	δ	π
a	0	NIL
b	4	a
c	1	d
d	5	b

Pseudo Code

Algorithm 1: Bellman-Ford algorithm

INITIALIZE(G, s)

for $i \leftarrow 1$ **to** $|V| - 1$ **do**

foreach $(u, v) \in E$ **do**

 RELAX(u, v)

- The number of iterations depends on the order in which edges are "relaxed" but it is at most $(n-1)$
- In the previous example 3 iterations were needed (which is the maximum since there are 4 vertices)

Complexity of Bellman-Ford

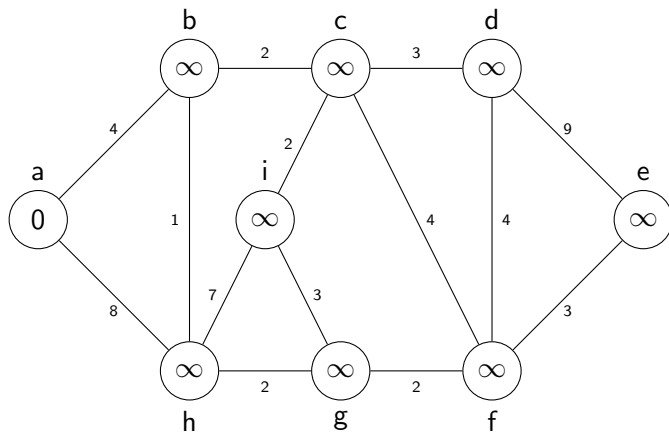
- The initialization is $O(|V|)$.
- the double loop is $O(|V| \cdot |E|)$.
- Therefore the total cost of the Bellman-Ford is $O(|V| \cdot |E|)$.
- Bellman-Ford is slower than Dijkstra's algorithm but it can handle negative weights.
- There is another formulation of Bellman-Ford using dynamic programming

Dijkstra's Algorithm

- Dijkstra's algorithm is another single source shortest path.
- It works when all weights are **positive**.
- We will see that it is faster than the Bellman-Ford algorithm.
- It maintains a set S of nodes whose shortest paths have been determined
- All other nodes are kept in a min-priority queue to keep track of the next node to process.

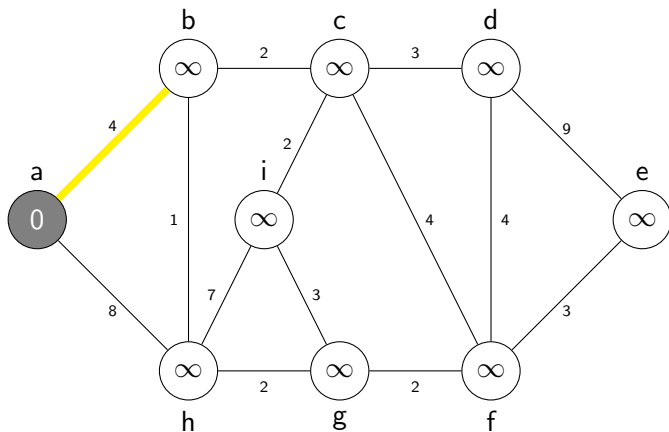
Example

a is source



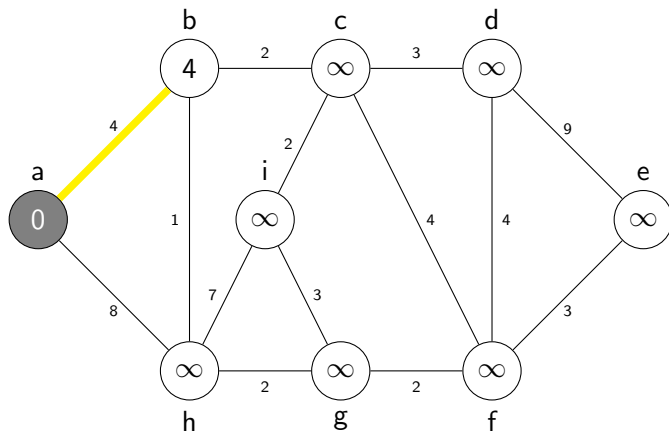
Example

neighbors of a



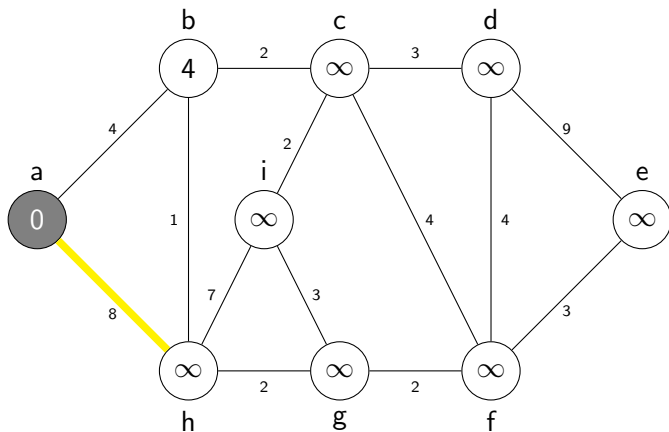
Example

neighbors of a



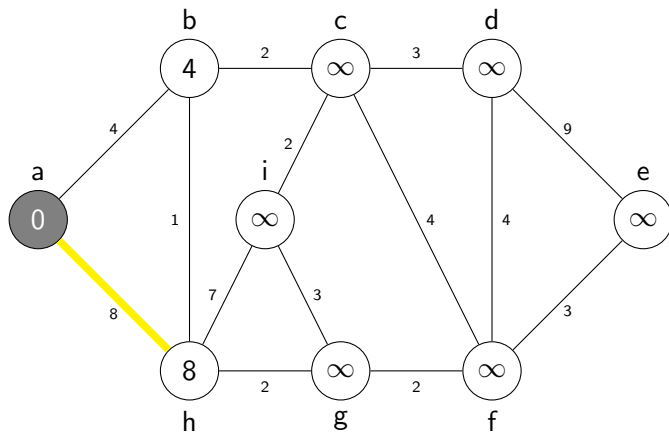
Example

neighbors of a



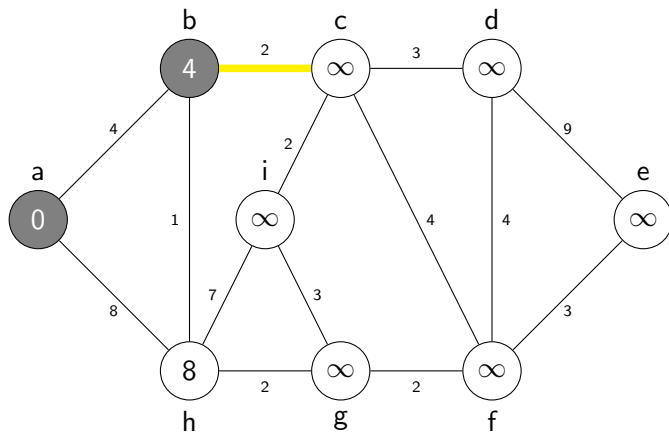
Example

neighbors of a



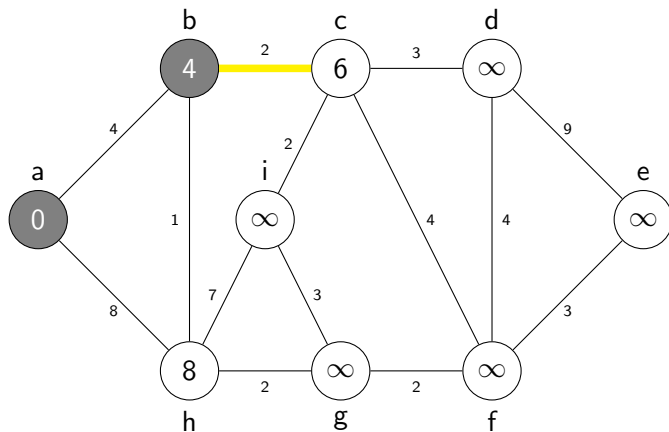
Example

neighbors of b



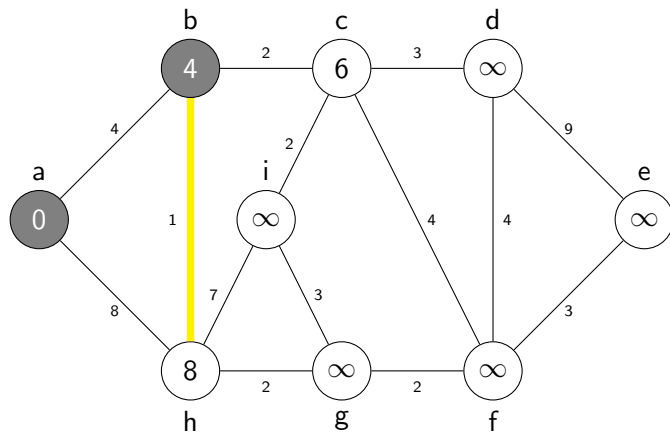
Example

neighbors of b



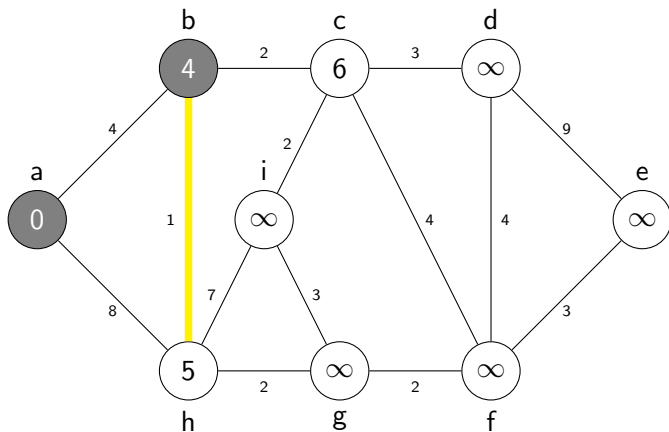
Example

neighbors of b



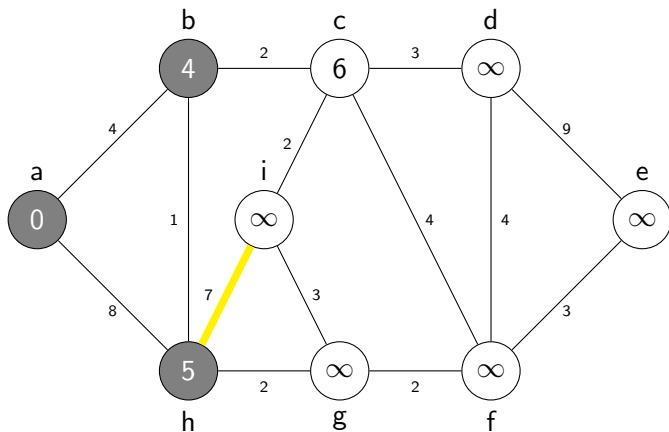
Example

neighbors of b



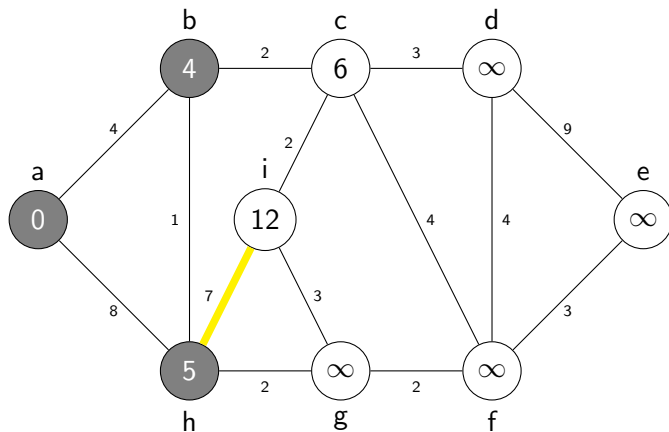
Example

neighbors of h



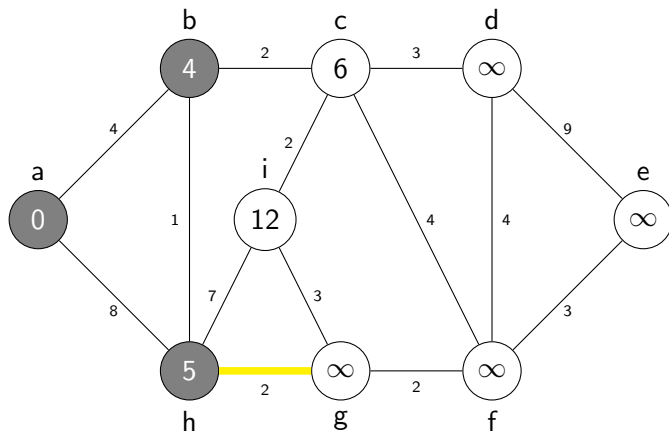
Example

neighbors of h



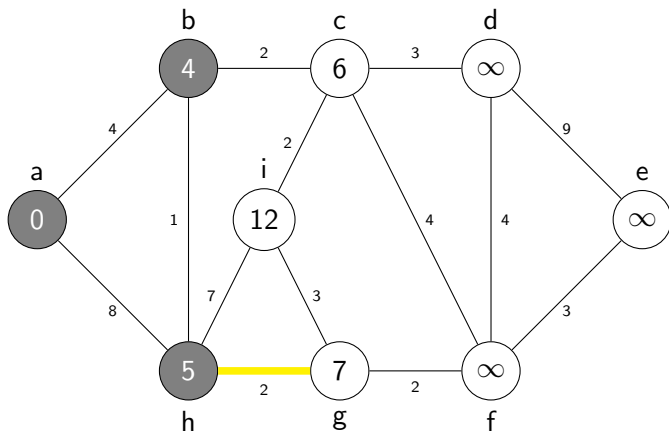
Example

neighbors of h



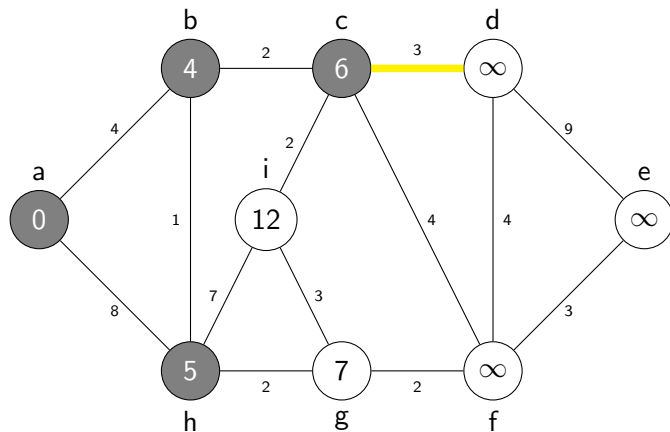
Example

neighbors of h



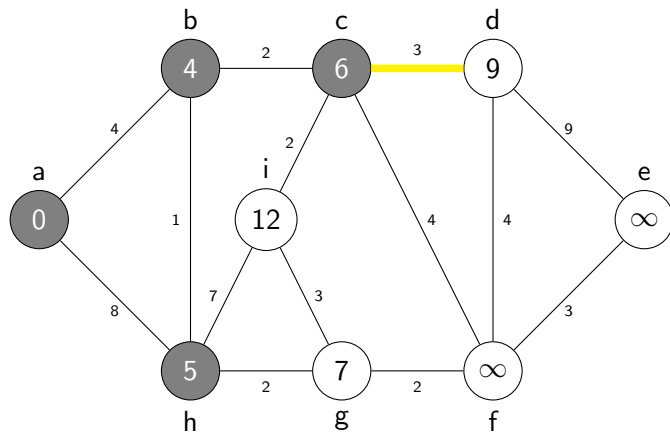
Example

neighbors of c



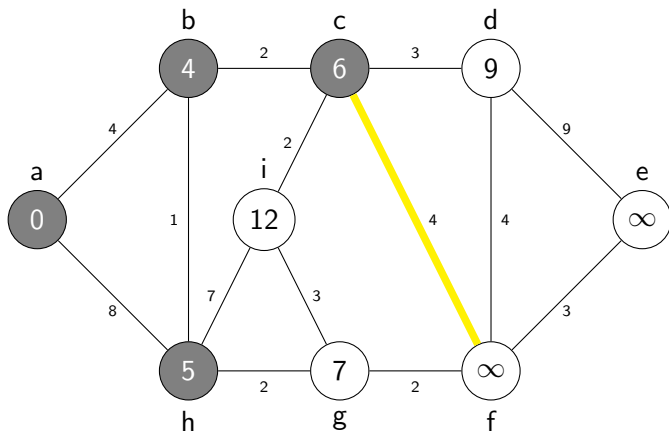
Example

neighbors of c



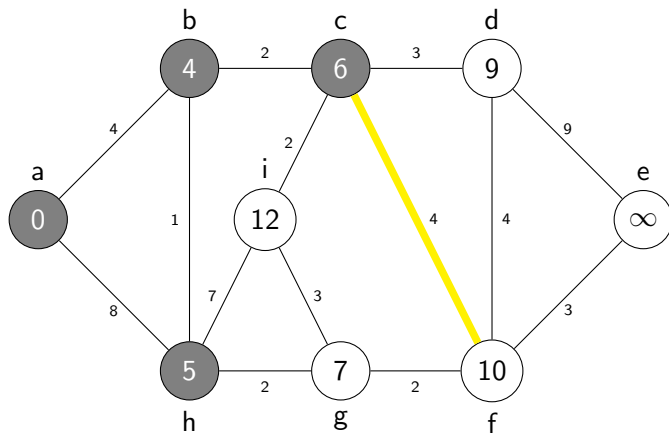
Example

neighbors of c



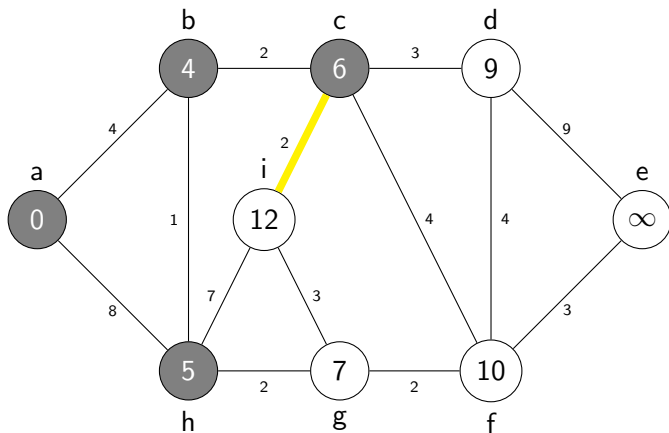
Example

neighbors of c



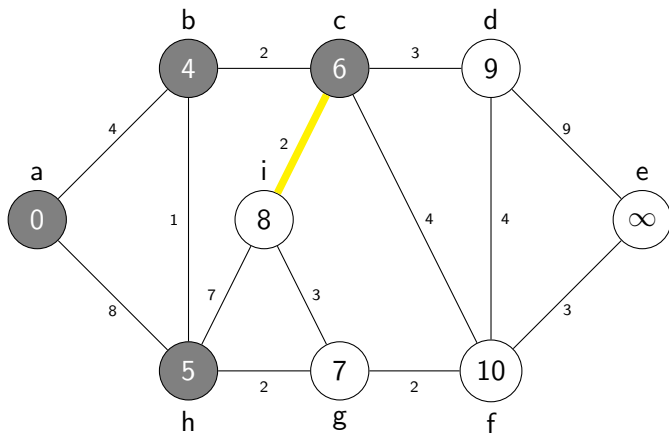
Example

neighbors of c



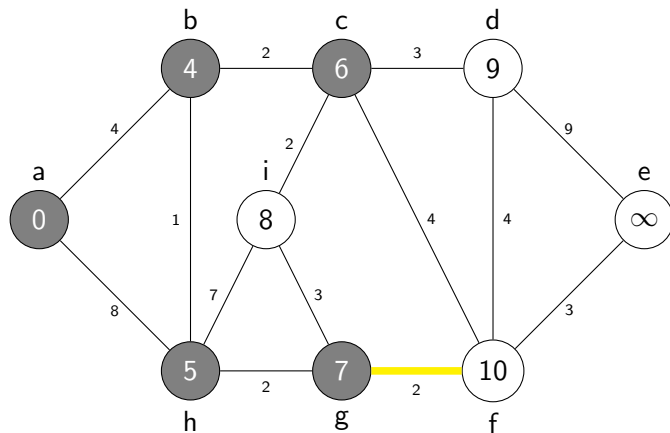
Example

neighbors of c



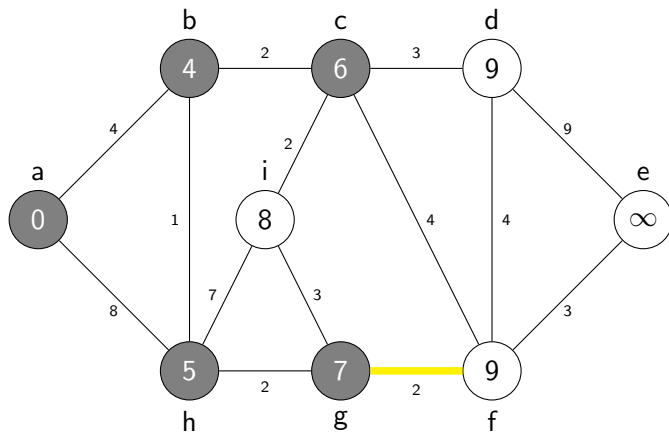
Example

neighbors of g



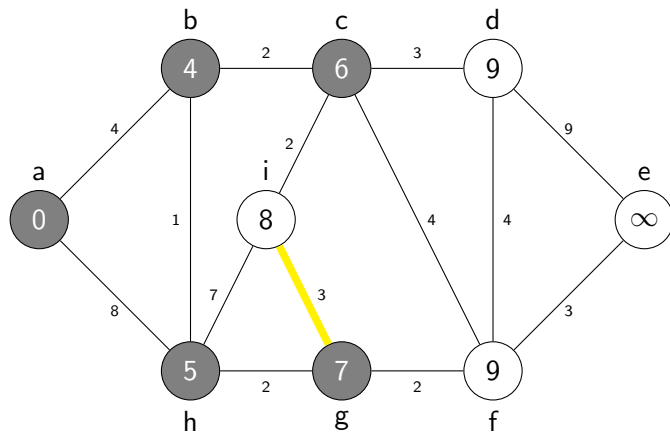
Example

neighbors of g



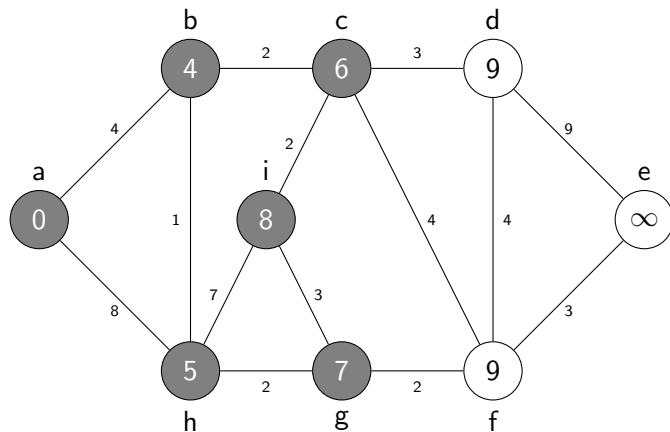
Example

neighbors of g



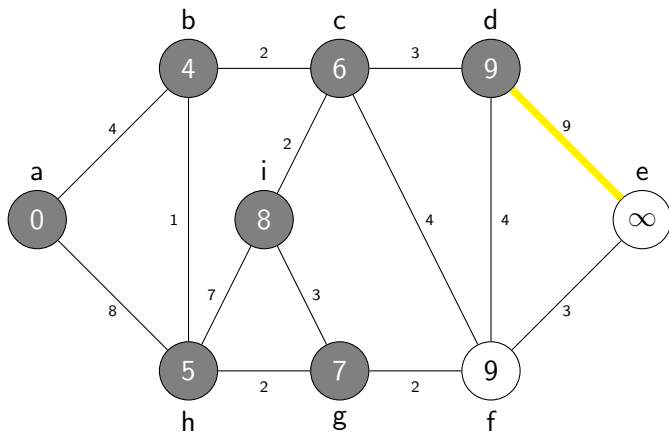
Example

neighbors of i



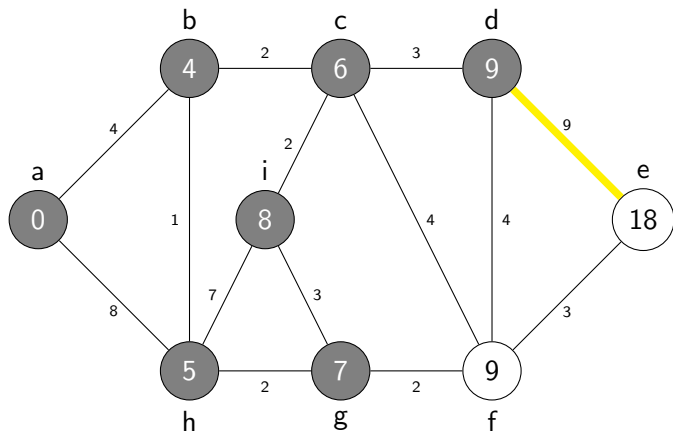
Example

neighbors of d



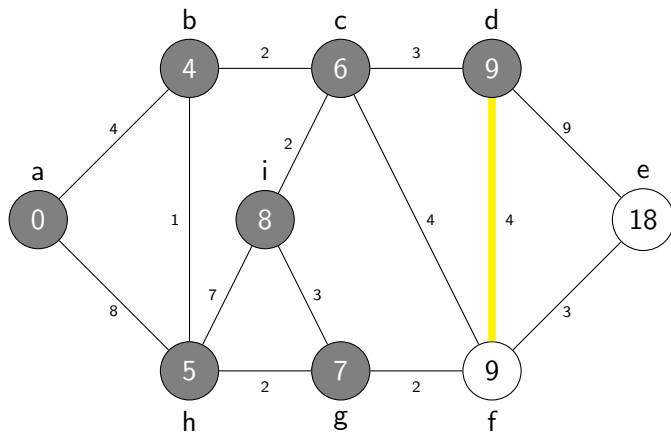
Example

neighbors of d



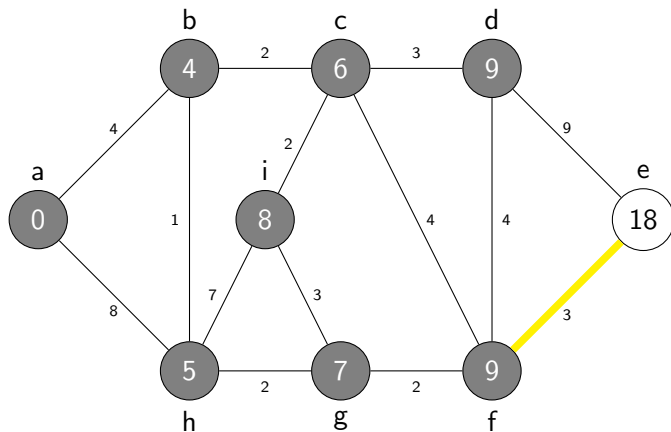
Example

neighbors of d



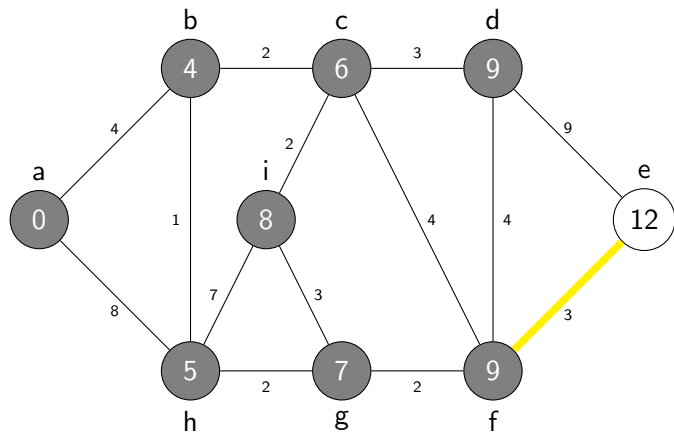
Example

neighbors of f



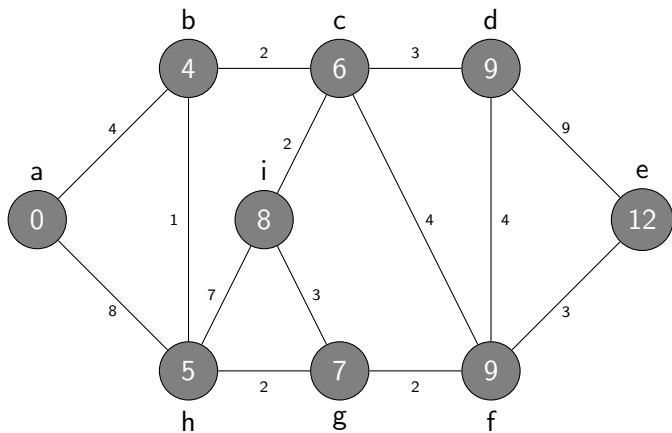
Example

neighbors of f



Example

Done

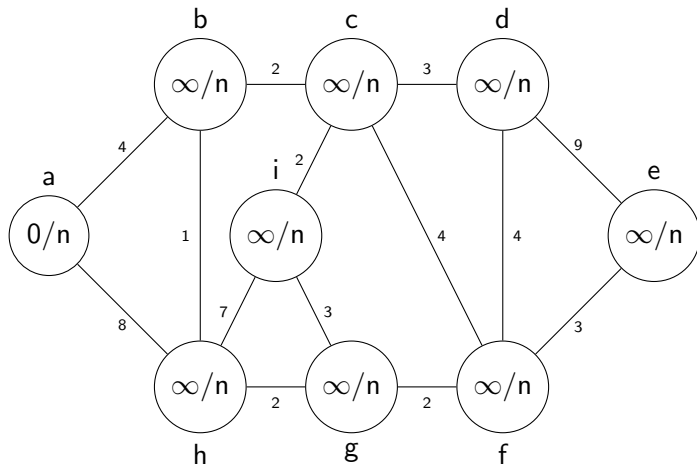


And the shortest PATH?

- The algorithm computes the shortest distance from the source to all other nodes.
- But in most situations we are interested in actual shortest path from the source to the destinations.
- This can be done by updating and saving the predecessor of each node.

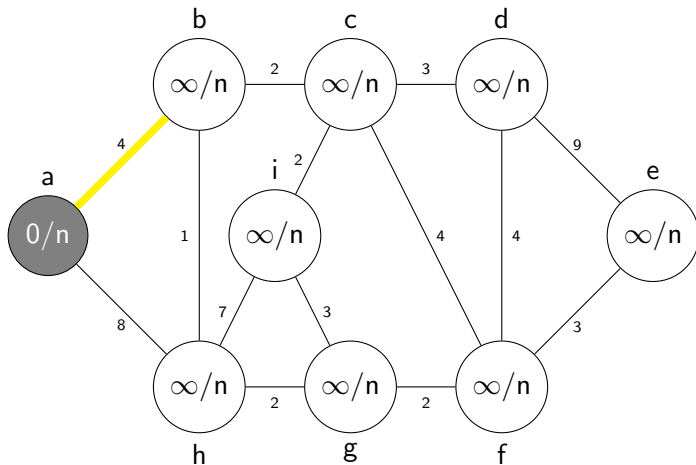
Example

a is source



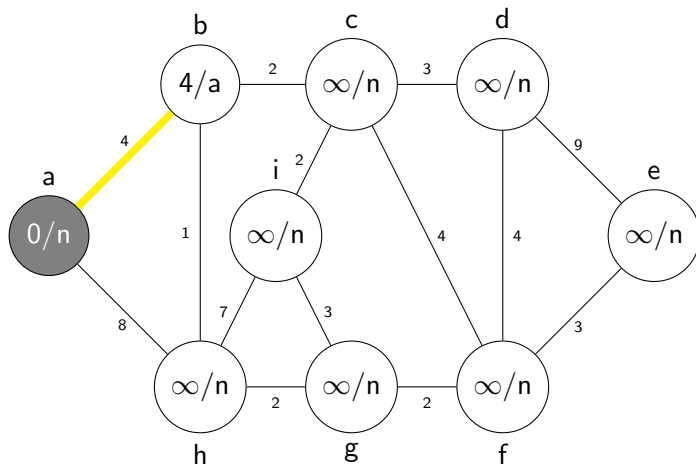
Example

neighbors of a



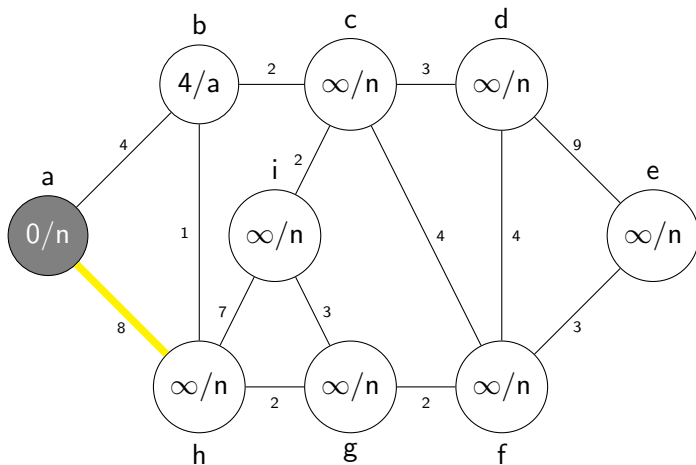
Example

neighbors of a



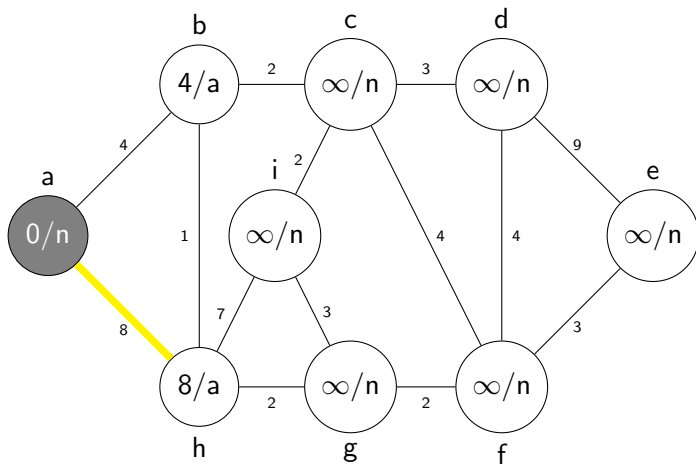
Example

neighbors of a



Example

neighbors of a

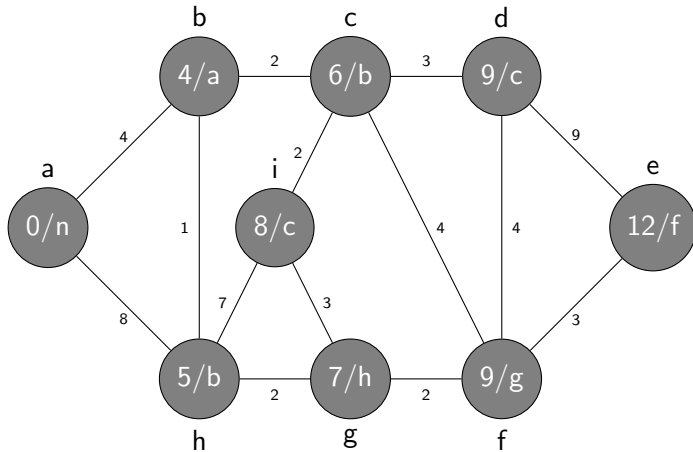


Example

ETC...

Example

Done



Recovering the shortest path

- By iterating backwards over the predecessors we can recover the shortest path.
- For example, consider node e
- $e.p = f, f.p = g, g.p = h, h.p = b, b.p = a$
- So the shortest path from a to e is a, b, h, g, f, e

Dijkstra Pseudo Code

```
DIJKSTRA( $G, s$ );  
INITIALIZE( $G, s$ )  
 $S \leftarrow \emptyset$   
 $Q \leftarrow V$   
while  $Q \neq \emptyset$  do  
     $u \leftarrow \text{EXTRACT-MIN}(Q)$   
     $S \leftarrow S \cup \{u\}$   
    foreach  $v \in \text{adj}[u]$  do  
        RELAX( $u, v$ )
```

Complexity

- The running time of Dijkstra's algorithm depends on the implementation of the queue.
- Using a min-heap on a sparse graph gives complexity of $O((|V| + |E|) \log |V|)$.
- This is because the while loop executes $|V|$ times. The extract-min is $O(\log |V|)$ for a cost of $|V| \log |V|$. The relax includes an key update which means $\log |V|$. Since each edge is relaxed at most once then the total is $|E|$ with a cost of $|E| \log |V|$.

Complexity

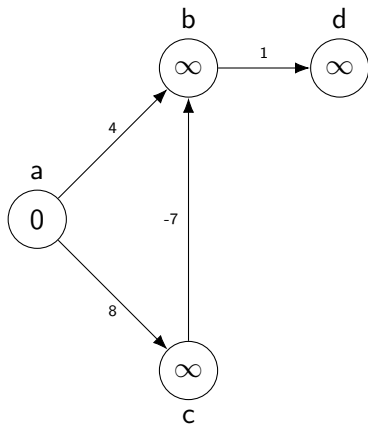
Let $n = |V|$ and $m = |E|$ and Q is a min-heap.

	DIJKSTRA(G, s);	
	INITIALIZE(G, s)	$O(n)$
1	$S \leftarrow \emptyset$	$O(1)$
2	$Q \leftarrow V$	$O(n)$
3	while $Q \neq \emptyset$ do	$O(n)$
4	$u \leftarrow \text{EXTRACT-MIN}(Q)$	$O(\log n)$
5	$S \leftarrow S \cup \{u\}$	$O(1)$
6	foreach $v \in \text{adj}[u]$ do	$O(\text{adj}[u])$
7	RELAX(u, v)	$O(\log n)$

The dominant term comes from the execution of line 7 $O(m)$ times(sum of adjacencies) for a total of $O(m \log n)$.

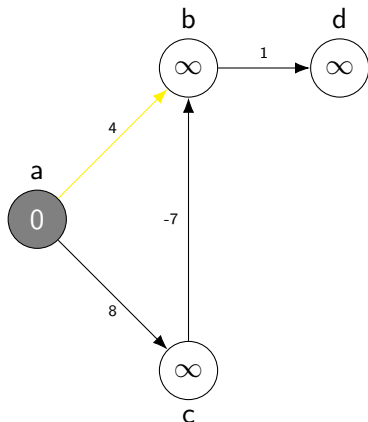
Negative weights counter example

a is source



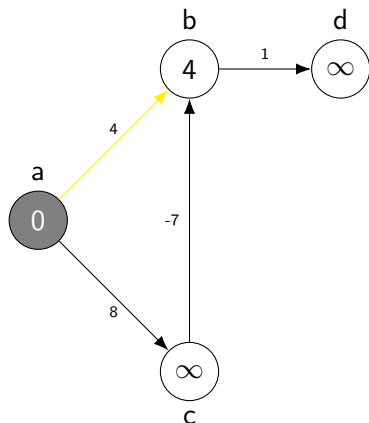
Negative weights counter example

neighbors of a



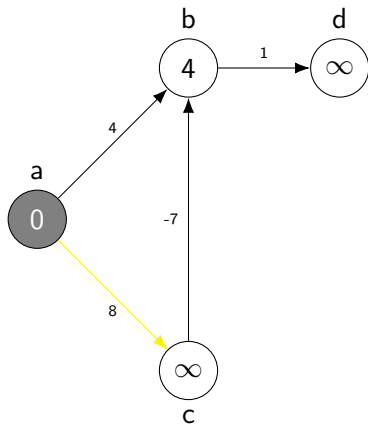
Negative weights counter example

neighbors of a



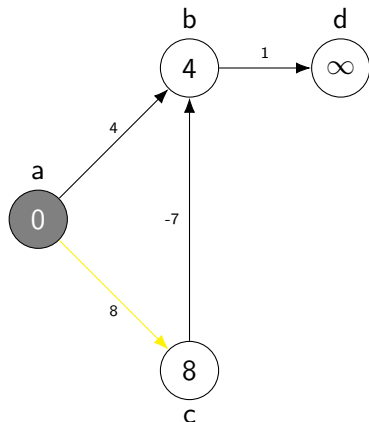
Negative weights counter example

neighbors of a



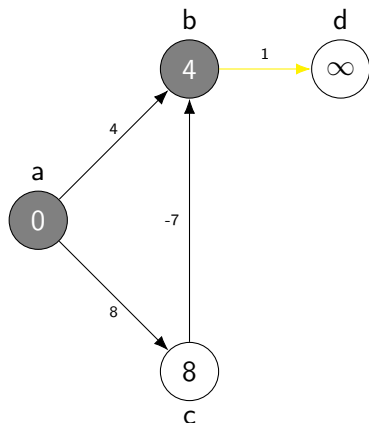
Negative weights counter example

neighbors of a



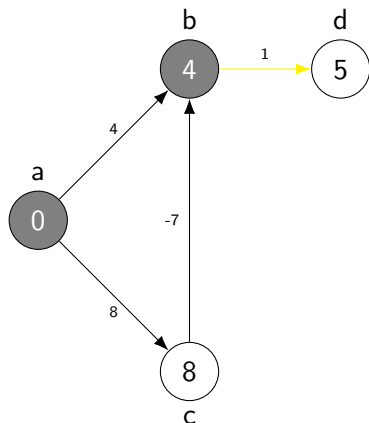
Negative weights counter example

neighbors of b



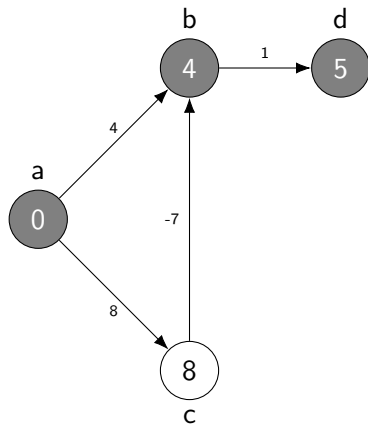
Negative weights counter example

neighbors of b

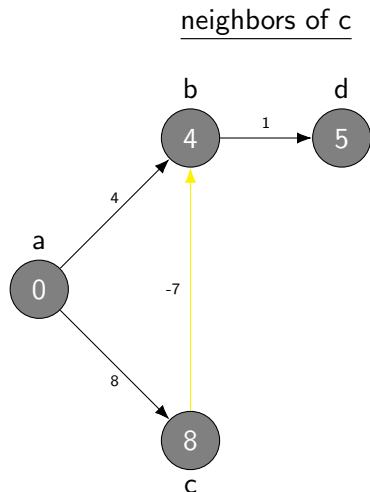


Negative weights counter example

neighbors of d



Negative weights counter example



Negative weights counter example

